

Real Estate Marketplace Platform

Comprehensive Technical Documentation, System Architecture & Applied Features Catalog

Attribute	Specification / Reference
Backend Repository	https://github.com/OmarCoder9/real-estate-market-place-api (NestJS, TypeScript, MongoDB, Cloudinary, JWT)
Frontend Repository	https://github.com/fatmayasser8/Front-end-OSC-Project (React 19, Vite)
Production / Live API	https://real-estate-market-place-api.vercel.app/api/v1
Documentation Date	March 2026 Full-Stack Engineering Specification

1. Executive Summary & Project Overview

The Real Estate Marketplace (branded as Nova Real Estate) is a modern, enterprise-grade, full-stack web platform engineered to bridge property buyers, verified sellers, and administrative moderators. The system replaces conventional, unstructured property classifieds with a trusted, highly interactive digital ecosystem featuring end-to-end seller identity verification, interactive geospatial discovery, rigorous admin moderation, and streamlined buyer-to-seller communication.

CORE MISSION: Key Project Philosophy: Building trust and transparency in real estate transactions through mandatory seller identity verification, strict listing quality moderation, accurate geolocation mapping, and secure dual-token authentication.

1.1 Core Problems Solved

- Eliminating Real Estate Scams & Fraud:** Sellers must submit official government identification or business credentials for administrative audit prior to publishing any property listings.
- Eliminating Outdated & Ghost Listings:** Every submitted property and listing modification enters an admin review queue. Properties can be marked as Sold or deleted with full cascading asset cleanup.
- Precision Location Intelligence:** Interactive Leaflet maps and MongoDB 2dsphere geospatial indexing enable buyers to search properties by radius (in km) around specific coordinates with automated reverse geocoding.
- Frictionless Multi-Channel Contact:** Buyers can instantly connect with property owners via direct WhatsApp click-to-chat, one-tap mobile calls, or email inquiries.
- Secure & Modern User Experience:** Features passwordless Google OAuth 2.0, 6-digit email OTP verification, timed password resets, and responsive dashboards tailored for buyers, sellers, and administrators.

2. Full-Stack System Architecture & Technology Stack

The platform adopts a decoupled Client-Server Architecture adhering to RESTful API principles. The client application is built with React 19 and Vite for blazingly fast rendering and responsive UX. The backend is powered by NestJS 11 with TypeScript, leveraging progressive modular architecture, dependency injection, robust DTO validation, and MongoDB Atlas persistence.

2.1 Frontend Technology Stack

Layer / Package	Technology Version	Architectural Role & Purpose
Core Framework	React 19.x + Vite	High-performance Single Page Application (SPA) architecture with fast HMR

Layer / Package	Technology Version	Architectural Role & Purpose
Routing & Guards	React Router DOM v7	Declarative client-side routing with ProtectedRoute and AdminRoute guards
Interactive Maps	Leaflet & React-Leaflet	Interactive map navigation, custom pin markers, and coordinates picker
Data Visualization	Recharts	Interactive Area charts & Pie charts in Admin Dashboard for KPI analytics
UI & Icons	Tailwind CSS + Bootstrap 5 + FontAwesome	Modern responsive layout, glassmorphism cards, and vector icon sets
Notifications & Modals	SweetAlert2	Rich, accessible popups, confirmation dialogs, and error alerts
Authentication	@react-oauth/google	Client-side Google One-Tap and OAuth 2.0 Sign-In integration
API Communication	Axios & Custom Fetch Wrapper	Automatic Bearer token injection, refresh token rotation, and multipart uploads

2.2 Backend Technology Stack

Component	Technology / Library	Role & Engineering Responsibility
Server Framework	NestJS 11.x (TypeScript)	Enterprise modular backend with Dependency Injection & Decorators
Database & ODM	MongoDB + Mongoose 9.x	NoSQL document store with 2dsphere compound geospatial indexing
Authentication	JWT (@nestjs/jwt) + Bcrypt	Access/Refresh tokens with bcrypt hashing and revocation flow
Security & Hardening	Helmet + CORS + Throttler	HTTP headers protection, origin whitelisting, strict OTP rate limiting
Data Validation	class-validator & class-transformer	Strict DTO schema validation, type casting, and malicious payload stripping
Media & Asset Store	Cloudinary SDK + Multer	Buffer streaming of property images and identity documents to Cloudinary
Mailing Service	Nodemailer + @nestjs-modules/mailer	Automated HTML emails using EJS templates for OTP and password resets
Reverse Geocoding	Axios + OpenStreetMap Nominatim	Converts geographical coordinates [lat, lng] into verified cities

Component	Technology / Library	Role & Engineering Responsibility
API Documentation	Swagger / OpenAPI 3.0	Interactive API playground at /api-docs with custom styled UI

2.3 Infrastructure & Cloud Deployment

Both the frontend and backend are configured for continuous deployment on modern cloud platforms. The NestJS backend supports both long-running Node.js process mode (using Express) and serverless deployment via Vercel through a specialized export handler in main.ts. Media assets are streamed directly from server memory (via streamifier) into Cloudinary cloud buckets, ensuring zero local disk persistence and high horizontal scalability.

3. User Roles & Access Control Matrix

The system enforces a hierarchical Role-Based Access Control (RBAC) model governed by three distinct roles: Buyer, Seller, and Administrator. Access to endpoints and frontend routes is strictly guarded by NestJS guards (AuthGuard, AuthRolesGuard) and React route wrappers.

Platform Feature / Capability	Buyer	Seller (Verified)	Administrator
Browse & Search Properties	Yes (Public)	Yes	Yes
Geospatial Map Search & Radius Filter	Yes (Public)	Yes	Yes
View Property Details & Seller Contact	Yes (Public)	Yes	Yes
Bookmark / Save to Favorites	Yes (Authenticated)	Yes	Yes
Submit Seller Verification (ID)	No	Yes (Required)	No
Publish & Edit Property Listings	No	Yes (If Approved)	Manage All
Seller Dashboard (Manage My Properties)	No	Yes	No
Review & Moderate Listings	No	No	Yes (Approve/Reject)
Review & Verify Seller ID Requests	No	No	Yes (Approve/Reject)
User Management & System Analytics	No	No	Yes (Full Control)

4. Exhaustive Applied Features Catalog

This section details every implemented feature across both the frontend web application and backend REST API, categorized by functional domain.

4.1 Authentication & Security Architecture

- **Role-Based Registration:** Users can register as either a Buyer or a Seller. Validates full name, email format, strong password, and unique Egyptian/international phone number format.
- **Bcrypt Password Encryption:** Passwords are never stored in plain text; hashed using bcryptjs with 10 salt rounds.
- **Two-Factor 6-Digit Email OTP Verification:** Upon registration, users are marked `isVerified: false` and receive a 6-digit OTP code via email. Code expires after 10 minutes.
- **Interactive OTP Input UI:** Frontend validation page features auto-focusing 6-box input fields, clipboard paste support, and invalid digit handling.
- **OTP Resend with Rate Limiting:** Users can request a new verification code with an enforced countdown timer; backend rate limiter limits OTP requests to 5 per minute per IP via `OtpThrottlerGuard`.
- **Dual-Token JWT Authentication:** Login generates a short-lived access token for stateless API requests and a long-lived refresh token. The refresh token hash is stored in MongoDB for validation.
- **Silent Refresh Token Flow:** Custom Axios/apiFetch interceptors detect expired access tokens (HTTP 401) and transparently request a new access token without interrupting user interaction.
- **Secure Logout & Token Revocation:** Logging out invalidates the refresh token on the server by setting `refreshTokenHash` to null, preventing token reuse.
- **Google OAuth 2.0 Integration:** Supports passwordless one-tap sign-in and sign-up using Google ID Tokens verified securely via `google-auth-library` on the server.
- **Forgot & Reset Password Workflow:** Users can request a password reset email containing a time-limited (15-minute) cryptographic hex token that opens a dedicated reset interface.

4.2 Seller Identity Verification Workflow

To establish marketplace trust and eliminate fraudulent postings, sellers must complete an identity verification process:

- **Identity Document Upload Modal:** A dedicated popup modal allows sellers to upload national identity cards, passports, or commercial registrations with drag-and-drop and image preview.
- **Cloudinary Identity Bucket:** Verification photos are securely uploaded via Multer and stored under a dedicated cloud folder (`real-estate/identities`).
- **Lifecycle State Machine:** Verification requests cycle through `PENDING` -> `APPROVED` or `REJECTED`. State is tracked on both the `ListingRequest` entity and the user profile.
- **Listing Creation Gatekeeper:** The frontend `useSellerVerification` hook and backend guards intercept property publishing attempts, blocking unverified sellers with a prompt to verify.

- **Admin Review & Detailed Feedback:** Admins can inspect the high-resolution identity document, approve it to grant seller publishing rights, or reject it with mandatory corrective feedback.
- **Resubmission & Self-Cancellation:** If rejected, the seller can view the admin's explanation and upload a corrected document. Sellers can also cancel pending requests at any time.

4.3 Property Listing Creation & Management (Sellers)

- **Comprehensive Multi-Attribute Creation Form:** Sellers can publish properties with title, detailed description, price, transaction type (Sale / Rent), property category (Apartment, Villa, Studio, Commercial), area in square meters, bedroom count, bathroom count, and amenities checklist.
- **Interactive Leaflet Map Location Picker:** Sellers can click or drag a custom map pin to pinpoint the property's exact coordinates [latitude, longitude].
- **Automated Reverse Geocoding:** Coordinates are sent to OpenStreetMap Nominatim reverse geocoding to automatically resolve the city and district name, preventing misspelled locations.
- **Multi-Image Cloud Upload:** Supports uploading up to 10 property images simultaneously with real-time thumbnail previews and individual delete options before submitting.
- **Admin Moderation Workflow:** Newly created listings automatically enter PENDING status and cannot be viewed by the public until reviewed and approved by an administrator.
- **Listing Update & Image Management:** Sellers can edit existing listings, upload new images, and selectively remove individual existing images. Cloudinary assets for deleted images are permanently removed.
- **Auto-Reversion on Edit:** If an approved listing is edited, its status automatically reverts to PENDING to ensure seller updates comply with marketplace guidelines.
- **Cascading Listing Deletion:** Deleting a listing cleans up all associated Cloudinary images, removes the listing ID from all users' favorites arrays, and purges the database record.
- **Seller Dashboard:** Provides sellers with a dedicated management portal displaying metrics, active properties, status badges (Approved, Pending Review, Rejected with reason), edit triggers, and delete buttons.

4.4 Property Search, Filtering & Geospatial Discovery

- **Full-Text & Keyword Search:** Search properties instantly by title keyword or address term with case-insensitive regex filtering.
- **Multi-Criteria Faceted Filtering:** Filter listings concurrently by Transaction Type (Sale/Rent), Property Type (Apartment/Villa/Studio/Commercial), Price Range (Min/Max), Area Range (Min/Max m²), Bedrooms, and Bathrooms.
- **MongoDB 2dsphere Geospatial Search:** Utilizes MongoDB's native \$near and \$geometry operators on [longitude, latitude] coordinates with \$maxDistance to find properties within a custom radius in kilometers.
- **Interactive Dual-View (List & Map):** Browsers can explore properties via responsive card grids or navigate an interactive Leaflet map populated with custom pins and property preview popups.

- **Saved User Location Preferences:** Saves the user's preferred search coordinates in local storage (novaPreferredLocation), enabling personalized repeat visits.
- **Fast Image Slicing:** Search queries return optimized payload sizes by slicing the image array to the primary thumbnail (\$slice: 1), reducing bandwidth and accelerating page rendering.

4.5 Property Details Page & Multi-Channel Contact

- **Multi-Image Photo Carousel / Gallery:** High-definition property images with navigation controls, thumbnail selector, and full-screen view.
- **Structured Property Specifications:** Badges displaying bedrooms, bathrooms, total square meters, price per unit, listing category, and property type.
- **Amenities Checklist:** Visual tags highlighting amenities such as Parking, Swimming Pool, 24/7 Security, Elevator, Central A/C, and Balcony.
- **Exact Map View:** Embedded interactive Leaflet map centered precisely on the property's coordinates.
- **Direct WhatsApp Click-to-Chat:** One-click button that pre-populates a WhatsApp message referencing the property title and link to the seller's phone number.
- **Direct Phone Call & Email Inquiries:** Integrated tel: and mailto: action buttons for direct communication.
- **Viewing & Popularity Counter:** Tracks property engagement and displays total favorites count.
- **Instant Favorite Toggle:** Heart icon button allowing logged-in buyers to save or remove the property from their personal wishlist.

4.6 User Favorites & Wishlist Management

- **Persistent Favorites in User Document:** Favorites are stored in the user's MongoDB document as an array of ObjectIds referencing the Listing collection.
- **Favorites Page:** Dedicated /favorites page displaying responsive cards of all saved properties with quick-view and remove actions.
- **Atomic Toggle API:** A single PATCH /users/favorites/:id endpoint toggles state atomically, preventing duplicate bookmarks and incrementing/decrementing favouritesCount.
- **Batch Clear Favorites:** Users can clear their entire saved list with a single confirmation prompt.

4.7 User Profile & Account Customization

- **Profile Dashboard:** Displays account information, verification badges, contact details, and role indicator.
- **Edit Profile Details:** Allows users to update their full name and phone number with instant backend validation.
- **Avatar Upload to Cloudinary:** Users can upload custom profile avatars. The backend uploads to real-estate/users and automatically purges old avatar assets from Cloudinary.
- **Avatar Deletion:** One-click option to revert avatar back to default placeholder.

- **Public Seller Profile:** Publicly accessible profile (/publicProfile/:id) enabling buyers to view a seller's rating, verified badge, and portfolio of active listings.

4.8 Administrator Control Center & Analytics

- **Executive KPI Dashboard:** Real-time summary metric cards calculating Total Users, Total Listings, Pending Moderations, Active Listings, Sold Listings, Total Sellers, New Users (Today & This Week), and New Listings (Today, Week & Month).
- **Interactive Recharts Visualizations:** Interactive Area Charts showing listing submission trends over time and Pie Charts displaying listing distribution by status and category.
- **Listing Moderation System:** Admins can browse paginated listings with status filters (Pending, Approved, Rejected) and search; preview property details, approve listings, or reject them with mandatory feedback reasons.
- **Seller Verification Requests Queue:** Admins review submitted identity documents in a modal, inspect applicant data, approve seller privileges, or reject requests with rejection reasons.
- **User Management Table:** Admins can browse all registered users, filter by role (Buyer, Seller), search by name/email/phone, and delete abusive accounts with cascading removal of their listings and requests.

5. Database Schema & Data Models Specification

The system utilizes MongoDB as its primary datastore, modeled via Mongoose schemas. Sensitive credentials (passwords, tokens, OTP hashes) are shielded using Mongoose transforms and select: false.

5.1 User Schema Specification

Field Name	BSON Type	Constraints / Index	Description
fullName	String	Required, Trim	Full legal name of the user
email	String	Required, Unique, Lowercase	Primary login identifier and OTP destination
password	String	Select: false	Bcrypt hashed password (stripped from JSON serialization)
phoneNumber	String	Unique, Sparse, Trim	Contact number for WhatsApp and calls
role	String (Enum)	Default: 'buyer'	'buyer' 'seller' 'admin'
isVerified	Boolean	Default: false	True once 6-digit email OTP is confirmed

Field Name	BSON Type	Constraints / Index	Description
otpHash / otpExpires	String / Date	Select: false	Hashed OTP code with 10-minute expiry timestamp
googleId	String	Unique, Sparse	Identifier for Google OAuth 2.0 SSO
userImage	String	Default: null	Secure Cloudinary URL of the user avatar
favorites	Array of ObjectId	Ref: 'Listing'	List of bookmarked property IDs

5.2 Listing Schema Specification

Field Name	BSON Type	Constraints / Index	Description
owner	ObjectId	Required, Ref: 'User'	ID of the seller who created the listing
title / description	String	Required, Trim	Property title (2-100 chars) and detailed description
price	Number	Required, Min: 0	Price in currency units (EGP / USD)
listingType	String (Enum)	Index: true	'sale' 'rent'
propertyType	String (Enum)	Index: true	'apartment' 'villa' 'studio' 'commercial'
areaSqMeters	Number	Required, Min: 1	Total area in square meters
bedrooms / bathrooms	Number	Min: 0 / Min: 1	Room count specifications
images	Array of String	Max: 10	Cloudinary secure URLs for uploaded property photos
amenities	Array of String	Default: []	Tags: Parking, Pool, Security, Elevator, etc.
location	Subdocument	2dsphere Compound Index	GeoJSON Point [lng, lat], address, and city
status	String (Enum)	Default: 'pending'	'pending' 'approved' 'rejected'
isAvailable / soldAt	Boolean / Date	Default: true	Availability flag and timestamp when sold

5.3 ListingRequest Schema (Seller Verification)

Field Name	BSON Type	Constraints / Index	Description
requester	ObjectId	Required, Ref: 'User'	Seller seeking identity verification
identityDocument	String	Required	Cloudinary URL of government ID / business license
status	String (Enum)	Default: 'pending'	'pending' 'approved' 'rejected'
rejectionReason	String	Default: null	Feedback provided by admin if rejected

6. REST API Endpoints Specification

All endpoints are prefixed with /api/v1 and return standardized JSON responses. The following table provides an exhaustive index of all available endpoints:

Method	Endpoint Route	Access Level	Purpose & Handled Action
POST	/api/v/users/auth/register	Public	Register new user and dispatch 6-digit email OTP
POST	/users/auth/verify-otp	Public (Throttled)	Verify email with 6-digit OTP code
POST	/users/auth/resend-otp	Public (Throttled)	Resend verification code with cooldown
POST	/users/auth/login	Public	Authenticate email & password, return JWT tokens
POST	/users/auth/google	Public	Sign in or sign up using Google OAuth ID token
POST	/users/auth/refresh-token	Public	Issue new access token using valid refresh token
POST	/users/auth/logout	Bearer Token	Revoke refresh token and logout
POST	/users/auth/forgot-password	Public	Request timed password reset email link
POST	/users/auth/reset-password	Public	Reset account password with valid reset token

Method	Endpoint Route	Access Level	Purpose & Handled Action
GET	/users/profile	Bearer Token	Get current user profile and verification status
PATCH	/users/profile	Bearer Token	Update user full name and phone number
POST	/users/profile-image	Bearer Token	Upload custom avatar to Cloudinary
DELETE	/users/profile-image	Bearer Token	Delete avatar from Cloudinary
GET	/users/favorites	Bearer Token	Retrieve all saved properties for user
PATCH	/users/favorites/:id	Bearer Token	Atomic toggle property in user favorites
DELETE	/users/favorites/clear	Bearer Token	Clear all favorites for current user
POST	/listings	Seller (Verified)	Create property listing (up to 10 images)
GET	/listings	Public	Search approved listings with filters & radius
GET	/listings/:id	Public	Get single approved property details
PATCH	/listings/:id	Seller (Owner)	Update property details and manage images
DELETE	/listings/:id	Seller (Owner)	Delete listing with cascading image cleanup
POST	/requests	Seller	Submit identity document for seller verification

7. Frontend Page Architecture & User Journeys

The React application is structured into intuitive, modular views that guide users seamlessly through browsing, property creation, account management, and administrative supervision:

- **Landing Page (/):** High-impact hero section showcasing platform branding, value propositions, featured property highlights, call-to-action buttons for buyers and sellers, and navigation to authentication.

- **Home & Exploration (/home):** The core property discovery engine. Integrates an interactive search bar, multi-category filter toggles, dual view (responsive card grid alongside an interactive Leaflet map), live coordinate tracking, and quick property preview modals.
- **Authentication Suite (/auth/*):** Consolidated tabbed interface offering clean transitions between Login, Registration, Google One-Tap, Forgot Password, and Reset Password views with responsive validation alerts.
- **OTP Validation (/validation):** Focused security verification screen with 6 auto-advancing digit inputs, real-time code validation, and resend timers.
- **Property Details (/property/:id):** Full-page property showcase featuring a multi-image carousel, detailed architectural specs, amenities checklist, exact Leaflet map location, and direct WhatsApp / Call buttons.
- **Sell / Edit Property (/sell-property & /edit-property/:id):** Multi-step listing creation and editing workspace. Includes interactive map pin placement with automatic reverse geocoding, multi-image upload with thumbnail removal, and comprehensive property attribute fields.
- **Seller Dashboard (/sellerDashboard):** Unified portal for property owners displaying active listings, moderation statuses (Approved, Pending Review, Rejected with reason popover), sold status toggles, and edit/delete actions.
- **Favorites (/favorites):** Wishlist gallery allowing buyers to inspect their bookmarked listings, navigate to property details, or clear all saved properties.
- **Profile (/profile):** Comprehensive account settings portal for managing user contact details, uploading/removing Cloudinary avatars, and initiating or checking identity verification status.
- **Admin Dashboard (/adminDashboard):** Executive management portal featuring Recharts KPI visual charts, user management with role filters, property listing approval/rejection queue, and seller identity verification document audit.

8. Security, Performance & Engineering Excellence

- **Zero Local File Storage:** File uploads are processed entirely in server RAM using Multer memory storage and streamed directly to Cloudinary using streamifier. This prevents disk exhaustion and enables serverless deployment on Vercel.
- **Data Sanitization & Injection Defense:** NestJS ValidationPipe configured with whitelist: true and forbidNonWhitelisted: true strips unrecognized payload keys and rejects malformed requests. Mongoose parameterized queries prevent NoSQL injection.
- **Strict Rate Limiting & Throttling:** Sensitive endpoints such as verify-otp and resend-otp are guarded by a custom OtpThrottlerGuard (5 requests per 60 seconds), neutralizing brute-force OTP attacks.
- **Security Headers via Helmet:** Configured with Content Security Policy, X-Frame-Options, X-Content-Type-Options, and HSTS headers to protect against clickjacking, MIME-sniffing, and XSS.
- **Optimized Database Indexing:** MongoDB 2dsphere compound indexing on location.coordinates, status, isAvailable, and listingType guarantees sub-millisecond query execution even across large property datasets.

- **Graceful Error Handling:** Centralized NestJS exception filters and React error boundaries ensure users receive friendly, actionable error messages while preventing internal server stack trace leaks.

9. Conclusion & Project Accomplishments

The Real Estate Marketplace project successfully delivers a complete, production-grade full-stack solution that adheres to modern software engineering standards. By combining an intuitive React frontend with a robust, modular NestJS backend, the platform establishes a safe, engaging, and high-performance marketplace for property buyers, sellers, and administrators alike.

SUMMARY CONCLUSION: The project stands as an exemplar of full-stack engineering excellence: incorporating modern authentication, geospatial location intelligence, multi-tier admin moderation, cloud media streaming, and rich interactive data visualizations.